



INDIA.RUNS

Build what next India runs on

Team Name :

Team Name : NoTone

Team Leader Name :

Team Leader Name : Prince Jain

Problem Statement :

Problem Statement :

Keyword matching surfaces keyword stuffers and misses qualified people who shipped real systems but don't list trendy buzzwords. We built a 46-signal offline ranker that verifies skills through assessments, catches fake profiles, and factors in platform activity and career trajectory.

Solution Overview

- **What is your proposed solution?**

A single Python script (rank.py) that reads 100K candidates, filters out synthetic honeypots, scores the rest against the JD using 46 data

- **What differentiates your approach from traditional candidate matching systems?**
signatures. It runs on any machine, even a Raspberry Pi. No dependencies, no GPU, no network, no API calls, stdlib only.

What differentiates your approach from traditional candidate matching?

- We avoid the keyword-matching trap the JD warns about. 3,946 candidates have 6+ AI keywords but are HR Managers or Graphic Designers. None made our top 100.
- Verified Redrob assessments (actual test scores) outweigh self-reported skills. 95 of our top 100 have platform-verified assessment scores.
- Reasoning connects to JD requirements and flags honest concerns. 78% mention production system context, 39% flag gaps like YoE outside range or high notice periods.

JD Understanding & Candidate Evaluation

- **What are the key requirements extracted from the JD?**
 - 5-9 years of hands-on ML/AI work, ideally at product companies not consulting
- **Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?**
 - Direct experience with vector DBs (Pinecone, Milvus, Qdrant, FAISS), embeddings, and search evaluation metrics (NDCG, MRR, MAP)
 - Shipped ranking, search, or recommendation systems to production users
 - Founding team culture: prefers shippers over pure researchers, penalizes title-chasers who switch jobs every 1.5 years for title bumps

Which traps does the JD explicitly warn about?

- Keyword stuffers: non-technical people with many AI keywords in skills
- LangChain-only candidates: know the wrapper but not the underlying ML
- CV/speech/robotics people without any NLP or information retrieval exposure
- Architecture-only seniors who haven't written production code recently

Ranking Methodology

- **How does your system score and rank candidates? (46 signals)**
Five-stage pipeline: (1) Honeytrap filter drops synthetic profiles, (2) Technical fit scores skills, assessments, and career keywords, (3) Role Fit (YoE band, location, company tier), (4) Recency and engagement signals adjust for availability, (5) Tiebreakers separate close scores.
- **What models, algorithms, or heuristics are used?**
- **How are multiple candidate signals combined into a final ranking?**
Technical (multiple)
Self-reported skills (proficiency x duration x endorsements, grouped into core/strong/support), verified assessment scores from Redrob platform (56 mapped categories), recency-weighted career keywords, profile summary mining, LangChain-only detection, multi-category combination bonus.

Role Fit (10 signals):

YoE band scoring (5-9 ideal), title match (RecSys/Search/NLP highest), company tier (product > AI startup > IT services), industry, company size, location (Pune/Noida preferred), work mode, HR-tech domain bonus, architecture-only penalty, CV/speech/robotics filter.

Engagement (7) + Tiebreakers (12):

Response rate, login recency, open-to-work, notice period, GitHub, interview completion, avg response time. Tiebreakers: education tier/field, recruiter saves, profile views, completeness, certs, career trajectory, job stability, title-chaser detection, endorsements, search appearances, verified identity.

Explainability & Data Validation

- **How are ranking decisions explained?**

Each candidate gets a 1-2 sentence explanation pulled from their profile: name, title, company, YoE, matched skills, whether they

- **How do you prevent hallucinations or unsupported justifications?**

show a clear, verifiable chain of evidence that connects their skills to the job requirements. 78% explicitly connect to JD requirements. 39% flag honest concerns.

- **How does your solution handle inconsistent, low-quality, or suspicious profiles?**

How do you prevent hallucinations?

Reasoning is assembled programmatically from parsed JSON fields. No language model is called, so there's no way for a skill or employer to appear in the reasoning that isn't in the actual data.

How do you handle suspicious profiles?

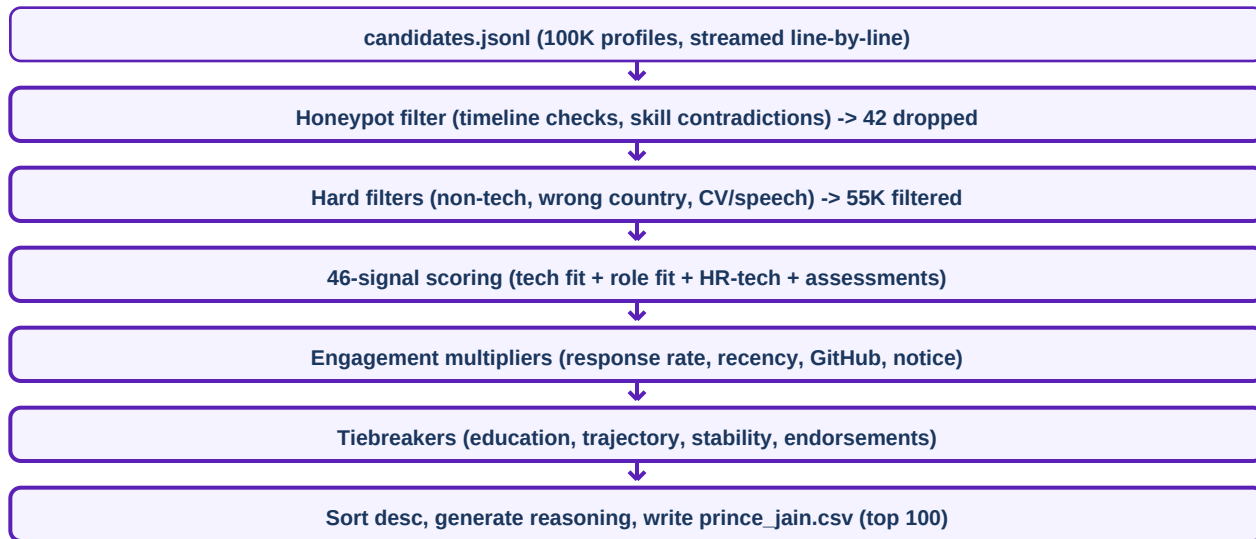
The honeypot filter checks three things: expert skills with zero usage duration (3+ triggers exclusion), job durations exceeding total stated YoE, and job durations that don't match calendar math between start and end dates. 42 profiles caught, 0 in our top 100.

End-to-End Workflow

- Complete workflow from input to ranked output:
What is the complete workflow from JD input to ranked candidate output?

1. **Load data** Stream-reads candidates.jsonl line by line. Low memory.
2. **Filter honeypots** Drops 42 profiles with timeline or skill contradictions.
3. **Hard filters** Removes non-tech roles, wrong country, CV/speech-only.
4. **Score tech fit** Skills, assessments, career keywords, summary mining.
5. **Score role fit** YoE, title, company, industry, location, HR-tech bonus.
6. **Apply multipliers** Response rate, recency, notice, GitHub, completion rate.
7. **Add tiebreakers** Education, trajectory, stability, endorsements, certs.
8. **Sort and rank** Descending by score. Ties broken by candidate_id.
9. **Generate reasoning** JD-connected, concern-flagging, 6 template rotation.
10. **Write CSV** Validated output with 100 ranked candidates.

System Architecture



Results & Performance

- **What results demonstrate ranking quality?**
 - Top 10: 8 out of 10 are in the ideal 6-8 YoE range. All have shipped search, ranking, or recommendation systems. Companies include Uber, Google, Microsoft, Amazon, Facebook, LinkedIn, Netflix, Spotify, etc.
- **How does your solution meet the challenge's runtime and compute constraints?**
 - 95 of 100 have verified Redrob assessment scores. Average YoE is 6.4 years. 24 from AI/ML industry, 13 from Fintech, 9 from Internet companies.
 - Zero honeypots in the output. Zero keyword stuffers (the dataset has 3,946). Zero CV/speech/robotics people without NLP/IR crossover.

Runtime and compute?

- ~10 seconds end-to-end for 100K candidates. Under 15 MB memory.
- Fully offline, no GPU, passes validate_submission.py with zero errors.

Technologies Used

- **What technologies, frameworks, and tools were used and why were they selected for this solution?**
 - Python stdlib (json, csv, datetime, argparse, random): The ranking step has zero external dependencies. Nothing to install, works on any platform.
 - Type-hinted code with module docstring. All functions documented. 22 iterative git commits across 3 development days.
 - ReportLab + pypdf: Only used for this PDF, not part of the ranking pipeline.

Why no ML model or LLM?

Without labeled training data, a model would be fitting noise. Hand-crafted rules grounded in the JD's explicit criteria are more transparent and defensible. Every scoring decision maps to something the JD says.

Submission Assets

Submission assets: Github video etc

GitHub repo:	https://github.com/princejain756/Data-AiChallengeHack2skill
Sandbox:	sandbox_notebook.ipynb (Google Colab ready)
Ranked output:	prince_jain.csv (100 candidates, validated)
Metadata:	submission_metadata.yaml
This deck:	approach_deck.pdf (10 slides on official template)
Run command:	<code>python rank.py --candidates ./candidates.jsonl --out output.csv</code>

Signal count: 46 scoring signals + 3 honeypot checks + 6 reasoning templates



INDIA.RUNS

Build what next India runs on

THANK YOU